

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

TA-IND-04

Análisis de Rendimiento Paralelo aplicado al Proyecto Fin de Curso

Asignatura: Aplicaciones Distribuidas (ISR-701)

Unidad: Unidad 4 – Cómputo Paralelo y Distribuido

Estudiante: Jeremy Ruperto Gaibor Rodríguez

Equipo PE-U4: Equipo C

PFC de referencia: Tienda Tech

Transformación foco: T1 – Filtrado y selección

Período académico: 2026–2027 PPA

Docente: Gleiston C. Guerrero-Ulloa, M.Sc.

Repositorio individual:

AQUI_VA_LA_URL_DE_TU_REPOSITORIO

Fecha: Agosto de 2026

1. Introducción y contexto

El procesamiento paralelo y distribuido permite repartir una carga de trabajo entre varias unidades de cómputo. Sin embargo, aumentar la cantidad de recursos no garantiza una reducción proporcional del tiempo de ejecución. En Apache Spark, la cantidad de executors constituye un parámetro relevante y una asignación inadecuada puede producir un uso poco eficiente de los recursos o un mayor tiempo de ejecución [1]. Asimismo, el ajuste de los parámetros de configuración de Spark constituye un problema de optimización que depende de las características de cada aplicación [2].

El presente informe analiza de manera individual los resultados obtenidos en la práctica PE-U4. En el experimento se implementaron cinco transformaciones equivalentes mediante pandas y PySpark sobre un conjunto sintético de 500.000 solicitudes y 14 columnas. Para cada configuración se realizaron cinco repeticiones y se utilizó la mediana como tiempo representativo. Antes de las repeticiones registradas se descartó una ejecución inicial de calentamiento.

La transformación seleccionada como foco individual es T1, correspondiente al filtrado y selección. Esta operación procesa los 500.000 registros de entrada y produce 33.117 registros como resultado.

Los datos utilizados son trazables al repositorio PE-U4 del Equipo C:

<https://github.com/ffarinangog2/pe-u4-spark-equipo-c>

Las mediciones adicionales de T1 con uno y dos executors se incorporaron en el commit 6b363ab. Los resultados se encuentran registrados en `resultados/tiempos_crudos.csv` y `resultados/tiempos_resumen.csv`.

El entorno documentado en el proyecto utiliza PySpark 3.5.0, Python 3.12.10, OpenJDK 17.0.20 y Windows 11. Spark fue ejecutado en modo Standalone con el master `spark://127.0.0.1:7077`. Cada executor fue configurado con un core y 512 MiB de memoria. Además, el número de particiones, el paralelismo por defecto y el máximo de cores se ajustaron al número de executors utilizado en cada configuración.

2. Análisis propio de los resultados de speedup

La comparación general entre pandas y PySpark utiliza el tiempo mediano de pandas y el tiempo obtenido por Spark con cuatro executors. El speedup se determina mediante:

$$S = \frac{T_{\text{pandas}}}{T_{\text{spark}}}$$

La Tabla 1 resume los resultados de las cinco transformaciones.

Cuadro 1: Resultados generales de PE-U4 con cuatro executors.

Transf.	T_{pandas} (s)	T_{spark} (s)	S	E
T1	0.2119435	0.3456639	0.6131	0.1533
T2	0.1171350	0.6874908	0.1704	0.0426
T3	0.2888583	0.8657363	0.3337	0.0834
T4	2.2658870	1.8100053	1.2519	0.3130
T5	1.5252129	1.3564848	1.1244	0.2811

Los resultados muestran que T4 y T5 alcanzaron un speedup superior a uno, mientras que T1, T2 y T3 fueron más rápidas mediante pandas. En el caso de T1 se obtuvo un speedup de 0,613.1, por lo que la ejecución de Spark con cuatro executors necesitó más tiempo que pandas.

Este resultado no es inesperado en un motor distribuido. Sen et al. muestran que distintas consultas de Spark pueden requerir niveles diferentes de paralelismo y que incrementar la cantidad de executors no garantiza siempre una reducción del tiempo de ejecución [1]. Por su parte, Cheng et al. estudian el ajuste de parámetros de Spark como un problema de optimización dependiente de la carga de trabajo [2].

2.1. Escalado de T1

Para estudiar el escalado de T1 se toma como referencia la ejecución Spark con un executor:

$$T_1 = 0,313.484.6 \text{ s.}$$

Los resultados obtenidos fueron los siguientes:

Cuadro 2: Escalado experimental de T1.

N	T_N (s)	$S(N)$	$E(N)$
1	0.3134846	1.0000	1.0000
2	0.8624843	0.3635	0.1817
4	0.3456639	0.9069	0.2267

Para dos executors, el speedup es:

$$S(2) = \frac{T_1}{T_2} \tag{1}$$

$$= \frac{0.3134846}{0.8624843} \tag{2}$$

$$= 0.3635. \tag{3}$$

Para cuatro executors:

$$S(4) = \frac{T_1}{T_4} \tag{4}$$

$$= \frac{0.3134846}{0.3456639} \tag{5}$$

$$= 0.9069. \tag{6}$$

En ninguna de las configuraciones se obtuvo una aceleración respecto de la ejecución con un solo executor. La configuración con dos executors presentó el peor resultado.

Sus cinco tiempos fueron 0,862.484.3 s, 0,375.832.5 s, 1,171.479.9 s, 0,275.453.9 s y 1,490.659.2 s. La diferencia entre el menor y el mayor valor evidencia una variabilidad considerable en esta configuración.

2.2. Contraste con la Ley de Amdahl

La Ley de Amdahl establece el límite del speedup que puede alcanzarse cuando una fracción p de un programa puede ejecutarse en paralelo [3]:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\text{máx}} = \frac{1}{1-p}. \quad (7)$$

El experimento no midió de forma independiente la fracción paralelizable p . Por ello, se utiliza $p = 1$ únicamente como una referencia ideal superior, es decir, como el caso en el que no existe fracción serial ni sobrecarga. Esta elección no implica afirmar que T1 sea completamente paralelizable durante una ejecución real.

Bajo esa referencia ideal:

$$S_{\text{teo}}(N) = N.$$

Para dos executors, la desviación relativa entre el valor medido y la referencia ideal es:

$$\Delta_2 = \frac{S_{\text{med}} - S_{\text{teo}}}{S_{\text{teo}}} \times 100 \quad (8)$$

$$= \frac{0.3635 - 2}{2} \times 100 \quad (9)$$

$$= -81.83 \%. \quad (10)$$

Para cuatro executors:

$$\Delta_4 = \frac{0.9069 - 4}{4} \times 100 \quad (11)$$

$$= -77.33 \%. \quad (12)$$

Por tanto, los resultados experimentales se encuentran claramente por debajo del límite ideal de Amdahl. El speedup real es incluso menor que uno para las dos configuraciones.

Esto no contradice la Ley de Amdahl. La expresión representa un límite teórico y no incorpora la sobrecarga introducida por un motor distribuido real. En Spark existe trabajo adicional relacionado con la creación, planificación y coordinación de las tareas.

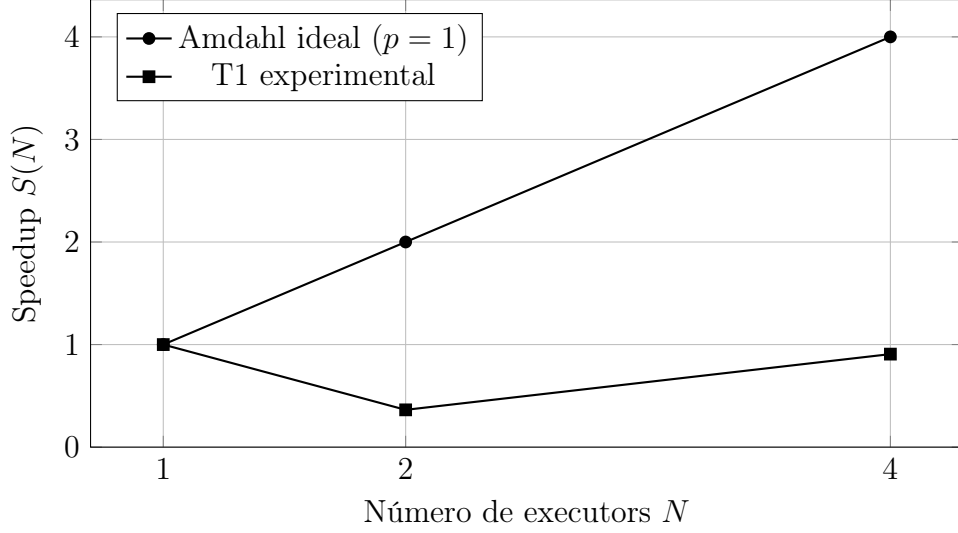


Figura 1: Speedup experimental de T1 frente al límite ideal de Amdahl. Elaboración propia.

La Fig. 1 muestra que el comportamiento experimental no sigue el incremento esperado en un escenario ideal. Con dos executors el rendimiento cae de forma considerable. Al utilizar cuatro existe una recuperación, pero el tiempo sigue siendo superior al registrado con un solo executor.

3. Diagnóstico de la fracción no escalable

La métrica de Karp-Flatt permite estimar experimentalmente una fracción serial a partir del speedup observado y del número de unidades de procesamiento [4]:

$$e = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1. \quad (13)$$

Para dos executors:

$$e_2 = \frac{\frac{1}{0.3635} - \frac{1}{2}}{1 - \frac{1}{2}} \quad (14)$$

$$\approx 4.5026. \quad (15)$$

Para cuatro executors:

$$e_4 = \frac{\frac{1}{0.9069} - \frac{1}{4}}{1 - \frac{1}{4}} \quad (16)$$

$$\approx 1.1369. \quad (17)$$

Los dos valores son superiores a uno. En estas condiciones no pueden interpretarse como una fracción serial física convencional, ya que el modelo está reflejando un escenario en el que el tiempo paralelo es peor que el tiempo de referencia.

Además, e disminuye de 4,502.6 con dos executors a 1,136.9 con cuatro. Por tanto, la pérdida observada no se comporta como una fracción serial constante. El resultado con dos executors está especialmente afectado por la variabilidad y la sobrecarga de la ejecución.

3.1. Sobrecarga observada

El primer mecanismo que puede relacionarse con los resultados es la planificación y coordinación del trabajo. Spark necesita preparar y distribuir las tareas antes de ejecutarlas, por lo que el nivel de paralelismo elegido puede afectar directamente al rendimiento. AutoExecutor estudia precisamente cómo distintas consultas Spark SQL requieren diferente cantidad de executors y cómo una selección inadecuada puede perjudicar el rendimiento [1].

El segundo mecanismo corresponde a la configuración de recursos. En el experimento, `spark.executor.instances`, `spark.cores.max`, `spark.sql.shuffle.partitions` y `spark.default.parallelism` cambian de acuerdo con N . Cheng et al. muestran que la configuración de Spark tiene una influencia suficiente sobre el rendimiento como para justificar métodos específicos de optimización de parámetros [2].

El tercer elemento es la variabilidad observada en las propias mediciones. Con dos executors se registraron tiempos comprendidos entre aproximadamente 0,275 s y 1,491 s. Esta dispersión indica que la sobrecarga efectiva de la configuración no se mantuvo constante durante las cinco repeticiones.

La eficiencia del paralelismo se determina mediante:

$$E(N) = \frac{S(N)}{N}. \quad (18)$$

Para dos executors:

$$E(2) = \frac{0.3635}{2} = 0.1817.$$

Para cuatro:

$$E(4) = \frac{0.9069}{4} = 0.2267.$$

Por tanto, la eficiencia fue aproximadamente 18,17 % con dos executors y 22,67 % con cuatro. La segunda configuración presenta una mejora respecto a la primera, pero ambas permanecen lejos de un aprovechamiento ideal del paralelismo.

4. Propuesta de integración al PFC

En el PFC Tienda Tech, el procesamiento distribuido se propone para el análisis histórico de pedidos, ventas y movimientos de inventario. No se plantea su uso dentro del flujo transaccional encargado de registrar directamente una compra, porque dicho proceso requiere respuestas rápidas y trabaja con una cantidad reducida de información por solicitud.

La integración se realizaría mediante procesamiento por lotes. Periódicamente se extraerían de PostgreSQL los registros históricos necesarios y se entregarían a un proceso Spark. Este proceso podría agrupar las ventas por producto y período, analizar el movimiento del inventario y generar indicadores de demanda.

La salida sería almacenada como información analítica preparada para consulta. El módulo de reportes utilizaría estos resultados para evitar el procesamiento completo del histórico cada vez que un usuario solicite información.

La decisión funcional soportada por esta integración sería la priorización de la reposición del inventario. A partir de los resultados podrían identificarse productos con alta demanda, bajo movimiento o necesidad de reposición.

Se propone una ejecución por lotes porque este procesamiento no necesita realizarse durante la compra de un cliente. Su ejecución podría realizarse periódicamente cuando se haya acumulado nueva información suficiente.

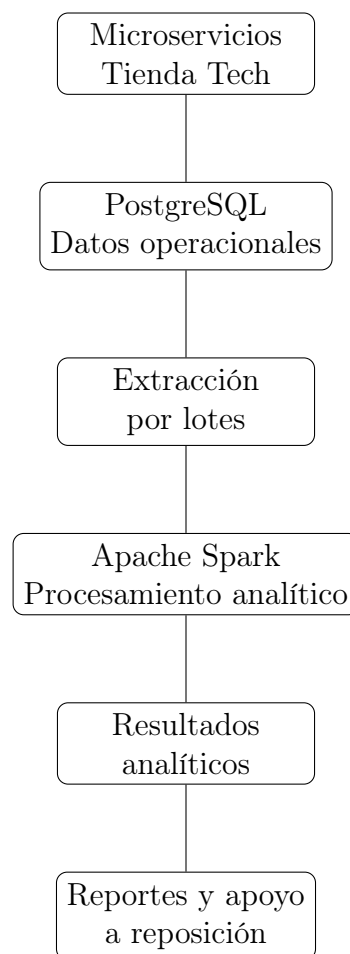


Figura 2: Propuesta de integración del procesamiento distribuido en Tienda Tech. Elaboración propia.

Como se observa en la Fig. 2, Spark se ubica después de la persistencia de los datos operacionales. De esta forma, el análisis se mantiene separado de los microservicios responsables de las operaciones transaccionales.

5. Dimensionamiento y viabilidad

El modelo propuesto para estimar la rentabilidad del procesamiento distribuido considera un tiempo secuencial:

$$T_{\text{sec}}(n) = \alpha n,$$

y un tiempo distribuido:

$$T_{\text{dist}}(n) = \beta + \frac{\gamma n}{N}.$$

El umbral se expresa mediante:

$$n^* = \frac{\beta}{\alpha - \frac{\gamma}{N}}, \quad \alpha > \frac{\gamma}{N}. \quad (19)$$

Los parámetros α , β y γ deben obtenerse mediante ajuste sobre mediciones realizadas con diferentes volúmenes de entrada. Los datos actuales de T1 corresponden a un único volumen de 500.000 registros, por lo que no permiten identificar de manera independiente los tres parámetros del modelo.

Por esta razón, con la evidencia disponible no se presenta un valor numérico de n^* . Calcularlo utilizando únicamente las configuraciones de uno, dos y cuatro executors implicaría utilizar variaciones de N como si fueran variaciones del volumen n , lo cual no corresponde al modelo de la Ec. (19).

Para completar el dimensionamiento sería necesario ejecutar T1 con distintos tamaños de entrada manteniendo controladas las demás condiciones experimentales. Gulino et al. estudian la predicción del tiempo de ejecución de aplicaciones Spark considerando, entre otros elementos, el tamaño de los datos y los recursos empleados [5]. De forma relacionada, Lattuada et al. analizan la asignación de recursos de aplicaciones Spark mediante modelos de rendimiento orientados a determinar la capacidad necesaria para alcanzar determinados objetivos de ejecución [6].

Mientras no se determine experimentalmente el umbral, para volúmenes similares a los evaluados se mantendría una solución basada en pandas o en consultas SQL, ya que T1 no presentó una ventaja de rendimiento mediante Spark.

Para una futura implementación, el procesamiento distribuido se ejecutaría como una tarea independiente por lotes y no como un servicio que deba permanecer activo de forma permanente. Entre los principales riesgos se encuentran la variabilidad del rendimiento, el costo de mantener infraestructura adicional, un crecimiento de datos insuficiente para justificar Spark y la sincronización de información procedente de varios microservicios.

6. Postura crítica y alternativas

Las mediciones disponibles indican que Spark no es actualmente la opción más eficiente para T1 con 500.000 registros. pandas obtuvo una mediana de 0,211.943.5 s, mientras que Spark con cuatro executors necesitó 0,345.663.9 s. Esto produjo un speedup de 0,613.1.

La evidencia experimental también muestra que aumentar la cantidad de executors no produjo una mejora progresiva. AutoExecutor señala que el nivel adecuado de paralelismo depende de la consulta ejecutada y que una cantidad inadecuada de executors puede

perjudicar el rendimiento o el uso de recursos [1]. Esto coincide con el comportamiento observado en T1.

Una primera alternativa es utilizar **pandas**. Para el volumen evaluado, esta opción obtuvo un tiempo menor y evita la coordinación requerida por un motor distribuido.

La segunda alternativa consiste en utilizar **PostgreSQL** para operaciones de filtrado y agregación cuando el volumen pueda procesarse de forma adecuada en el servidor de base de datos. Esta opción evita introducir un componente adicional en la arquitectura para cargas que aún no requieren procesamiento distribuido.

Spark se reservaría para escenarios donde el crecimiento de los históricos de pedidos, ventas e inventario supere la capacidad conveniente de una solución de un solo nodo. Antes de adoptarlo deberían realizarse nuevas pruebas con diferentes volúmenes para comprobar que el paralelismo compensa su sobrecarga.

Por tanto, Spark puede considerarse una alternativa para el componente analítico futuro de Tienda Tech, pero las mediciones actuales no justifican utilizarlo para T1 bajo las condiciones evaluadas.

7. Conclusiones

La transformación T1 no obtuvo una aceleración al incrementar el número de executors. Tomando como referencia Spark con un executor, el speedup fue 0,363.5 con dos executors y 0,906.9 con cuatro. Ambos valores fueron inferiores a uno, por lo que agregar recursos no redujo el tiempo respecto de la configuración base.

La métrica de Karp-Flatt produjo valores de 4,502.6 para dos executors y 1,136.9 para cuatro. Estos resultados, junto con la dispersión de los tiempos observada con dos executors, indican que la ejecución estuvo fuertemente afectada por costos adicionales y que la pérdida de rendimiento no puede explicarse únicamente mediante una fracción serial constante.

Finalmente, Spark tendría mayor sentido en Tienda Tech como componente de procesamiento por lotes para históricos de pedidos, ventas e inventario que como parte del flujo transaccional. Sin embargo, antes de adoptarlo debe determinarse experimentalmente el volumen a partir del cual la distribución resulta rentable mediante nuevas mediciones con distintos tamaños de entrada.

8. Declaración de uso de inteligencia artificial generativa

Se utilizó ChatGPT como herramienta de apoyo para organizar el contenido del informe de acuerdo con la rúbrica, revisar la redacción técnica, comprobar operaciones matemáticas y apoyar la estructuración de tablas y figuras.

Los datos experimentales no fueron generados mediante inteligencia artificial. Los tiempos utilizados proceden de las ejecuciones registradas en el repositorio PE-U4 del Equipo C. Las mediciones adicionales de T1 con uno y dos executors se encuentran identificadas en el commit declarado en este informe.

Las conclusiones fueron elaboradas a partir de los resultados experimentales registrados y las referencias académicas utilizadas se incorporaron para respaldar únicamente las afirmaciones relacionadas con sus respectivos resultados y ámbitos de estudio.

Referencias

- [1] R. Sen et al., “AutoExecutor: Predictive Parallelism for Spark SQL Queries,” *Proceedings of the VLDB Endowment*, vol. 14, n.º 12, págs. 2855-2858, 2021. DOI: 10.14778/3476311.3476362.
- [2] G. Cheng, S. Ying y B. Wang, “Tuning Configuration of Apache Spark on Public Clouds by Combining Multi-Objective Optimization and Performance Prediction Model,” *Journal of Systems and Software*, vol. 180, pág. 111028, 2021. DOI: 10.1016/j.jss.2021.111028.
- [3] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,” en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, ACM, 1967, págs. 483-485. DOI: 10.1145/1465482.1465560.
- [4] A. H. Karp y H. P. Flatt, “Measuring Parallel Processor Performance,” *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: 10.1145/78607.78614.
- [5] A. Gulino, A. Canakoglu, S. Ceri y D. Ardagna, “Performance Prediction for Data-Driven Workflows on Apache Spark,” en *2020 28th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, IEEE, 2020, págs. 1-8. DOI: 10.1109/MASCOTS50786.2020.9285944.
- [6] M. Lattuada, E. Barbierato, E. Gianniti y D. Ardagna, “Optimal Resource Allocation of Cloud-Based Spark Applications,” *IEEE Transactions on Cloud Computing*, vol. 10, n.º 2, págs. 1301-1316, 2022, Early access 2020. DOI: 10.1109/TCC.2020.2985682.